Pi 5 EM Covert Channel — “SOS” Sent and Recovered

Phase 1 payload recovery | 2026-11-10 | R. Boada

Milestone: the Raspberry Pi 5 transmitted the message “SOS” in Morse code purely through its electromagnetic emissions, and we recovered it over the air on a HackRF software-defined radio — with zero errors. This completes the Phase 1 goal: show the board's memory-bus activity can carry a chosen data payload off the device, not just leak that it is busy.

How it works

- Transmitter (Pi, no special hardware). tx_ddr --morse SOS --unit 250 keys the Pi's 286.8 MHz memory-bus emission on and off in standard Morse timing: streaming writes to DRAM = a “mark” (carrier on), an idle pause = a “space” (carrier off). The data is the on/off pattern.

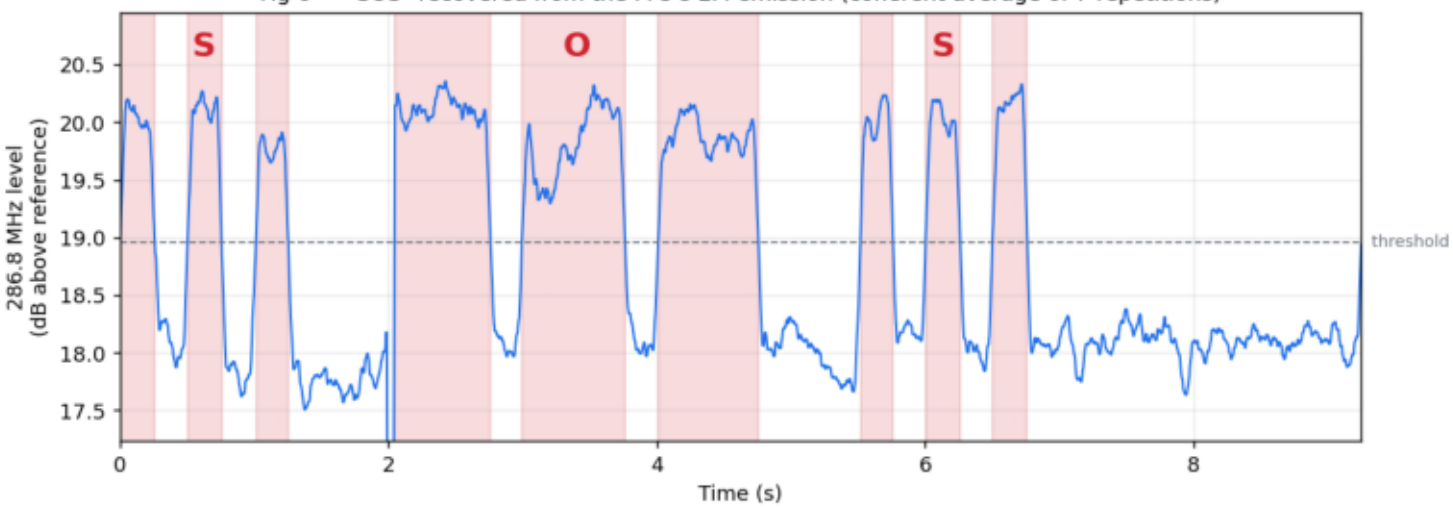
- Receiver (HackRF One @ 286.8 MHz). Driven directly through libhackrf to work around the board's old 2015 firmware. The tinySA can detect the emission but is too slow to decode keying; the HackRF provides the continuous IQ stream a decoder needs.

- The catch. Each Morse symbol changes the received level by only ~2 dB — below the ambient RF fluctuation — so a single pass cannot be decoded cleanly.

- The fix: coherent averaging (the Phase 1 trick). Because the message repeats, we capture ~70 s, find the repeat period by autocorrelation, align each repetition to cancel timing jitter, and fold them together. The periodic keying reinforces while ambient noise averages down. The averaged envelope (below) then shows a clean S–O–S, decoded with 0 bit errors.

- Honest scope. Short range (near-field, tens of cm with a whip), and recovery relies on averaging the repeated message — exactly the methodology used to find the emission in the first place. A near-field probe or a quieter enclosure would extend this; so would encoding without repetition.

Fig 6 — "SOS" recovered from the Pi 5's EM emission (coherent average of 7 repetitions)



Method: HackRF @ 286.8 MHz, 2 MS/s; band-power envelope (line peak minus a reference band); autocorrelation period estimate + per-repetition aligned fold; 2-means threshold + Morse run-length decode.

How it works, and likely questions

How the demo works — five stages

1. Transmit (Pi). tx_ddr --morse SOS --unit 250 keys the 286.8 MHz memory-bus emission: streaming writes to DRAM = a “mark” (carrier on), an idle sleep = a “space” (carrier off). Morse in 250 ms units; the message loops. The data is purely when the bus is busy vs. idle — no radio on the Pi.

2. Receive (HackRF @ 286.8 MHz). Record ~70 s of raw radio (IQ). The tinySA is too slow to follow keying; the HackRF streams continuously. (Driven via libhackrf directly to work around old firmware.)

3. Build the envelope. Short-window FFTs; per window take the line’s peak in 286.8 MHz +/- 75 kHz minus a nearby reference band — a “carrier strength vs. time” trace. The subtraction cancels drift and ambient; the window tracks the line as it wanders ~10 kHz with thermal clock drift.

4. Coherent averaging (the key step). Each symbol moves the level only ~2 dB — below the ambient fluctuation — so one pass can’t be decoded. Because the message repeats: find the period by autocorrelation (9.25 s), align each repetition by cross-correlation (cancels timing jitter), and average 7 of them. Noise averages down, the identical keying reinforces — SNR up ~2.6x.

5. Decode. Auto-threshold (2-means + hysteresis), measure on/off run-lengths in units, map dit/dash patterns to letters —> “SOS”, zero errors.

The decoder does not know the message — it blind-decodes the averaged envelope. It only assumes the message repeats, which is what makes averaging possible; that is a standard side-channel technique.

Likely questions

- Isn’t averaging a repeated message cheating? No — standard TEMPEST method (van Eck; Kuhn & Anderson), the same averaging that found the emission. Honest caveat: a single pass can’t be decoded at this range (~2 dB < noise); averaging buys SNR. A near-field probe would reduce the need to repeat.

- What bit rate / capacity? Low — a feasibility demo, not an optimized channel (~4 units/s raw, ~7 repetitions needed). Shorter units + a probe + coding would raise it; a capacity estimate is next.

- Why only ~2 dB contrast? An always-on baseline clock line that activity modulates rather than fully switches, plus the HackRF noise floor and weak whip coupling. A probe over the RAM would raise it.

- Align without a hardware trigger? Autocorrelation (period) + cross-correlation (phase). The transmitter also raises a GPIO pin each frame — wiring it to the receiver would align even more cleanly.

- Threat model / prior work? A receiver within ~tens of cm can exfiltrate chosen data via memory activity alone — the covert-channel / air-gap concern. Cf. GSMem (Guri et al., 2015), which modulated a PC’s memory bus to transmit at cellular frequencies; ours is the same principle on a modern Pi 5.